



UNIVERSITÀ DEGLI STUDI DI ROMA "LA SAPIENZA"
FACOLTÀ DI INGEGNERIA DELL'INFORMAZIONE,
INFORMATICA E STATISTICA

CORSO DI LAUREA TRIENNALE IN INFORMATICA

Relazione di progetto

MyFileTransferApp

Luglio 18, 2024

Candidato:

Lorenzo Sabatino, [REDACTED]@studenti.uniroma1.it

Matricola [REDACTED]

Docente:

Prof. Emiliano Casalicchio

Anno Accademico 2023-2024

Indice

1	Introduzione	2
2	Struttura del Programma	2
2.1	Architettura client-server	2
2.2	Gestione dei file da inviare/ricevere	2
2.3	Gestione di accessi concorrenti sui file	3
3	Conclusioni	3

1 Introduzione

Il presente documento descrive le scelte progettuali e implementative effettuate nello sviluppo di un sistema di trasmissione di file tra client e server. Verranno discusse le tipologie di socket utilizzate, la gestione delle comunicazioni e il supporto per le richieste concorrenti.

2 Struttura del Programma

2.1 Architettura client-server

Il progetto è stato sviluppato utilizzando un'architettura client-server, dove il server gestisce le richieste dei client e fornisce i servizi richiesti. Questa struttura permette di gestire più client simultaneamente e di fornire risposte in modo efficiente.

Le comunicazioni tra client e server sono state gestite attraverso l'uso di socket. Il server ascolta su una porta specifica e accetta connessioni dai client. Una volta stabilita la connessione, il server può inviare e ricevere dati dal client. In particolare, per la comunicazione tra client e server, sono state utilizzate socket di tipo TCP (*Transmission Control Protocol*). Questa scelta è stata motivata dalla necessità di garantire una comunicazione affidabile e orientata alla connessione tra le parti. Per gestire richieste concorrenti, il server crea un nuovo thread per ogni connessione in arrivo. Questo permette di gestire più client simultaneamente senza bloccare il server.

Il server è stato configurato per rimanere in ascolto su una porta specifica e per accettare le connessioni in entrata dai client. Una volta stabilita la connessione, il server e il client possono scambiarsi i file bidirezionalmente. Le principali funzioni implementate nel server sono: creazione del socket, binding del socket, ascolto delle connessioni, accettazione delle connessioni e gestione delle richieste.

Il client invia richieste al server e attende risposte. Le principali funzioni implementate nel client sono: creazione del socket, connessione al server, invio della richiesta e ricezione della risposta.

2.2 Gestione dei file da inviare/ricevere

La gestione dei file è stata implementata per consentire il trasferimento di file tra client e server. I path locali e remoti sono gestiti in modo da garantire la corretta memorizzazione dei file.

I path su cui devono essere memorizzati i file vengono creati se non esistono già, garantendo che la struttura delle directory necessarie sia presente prima del trasferimento del file.

I file vengono letti in modalità binaria e inviati tramite il socket utilizzando uno stream di byte (`byte stream`). I file vengono ricevuti in modalità binaria e scritti sul disco, assicurando che l'intero contenuto sia correttamente salvato.

Sono stati gestiti correttamente anche tutti gli errori relativi a file non esistenti, path errati e file vuoti.

2.3 Gestione di accessi concorrenti sui file

Per garantire la coerenza dei dati durante l'accesso concorrente ai file, sono stati utilizzati i mutex. I mutex (mutual exclusion) sono meccanismi di sincronizzazione utilizzati per evitare che più thread accedano simultaneamente a una risorsa condivisa, in questo caso i file.

Il mutex viene acquisito prima di eseguire operazioni sul file e rilasciato dopo che le operazioni sono state completate. Questo assicura che solo un thread possa accedere al file alla volta, evitando conflitti e corruzione dei dati.

3 Conclusioni

Il sistema di comunicazione tra client e server sviluppato utilizza socket TCP per garantire affidabilità e sicurezza nelle trasmissioni. La gestione delle richieste concorrenti è stata implementata tramite thread, permettendo al server di gestire più connessioni simultaneamente. Questa architettura assicura una comunicazione efficiente e scalabile tra le parti coinvolte.